30.1. Reliability

Reliability is an important property of any serious database system, and PostgreSQL does everything possible to guarantee reliable operation. One aspect of reliable operation is that all data recorded by a committed transaction should be stored in a nonvolatile area that is safe from power loss, operating system failure, and hardware failure (except failure of the nonvolatile area itself, of course). Successfully writing the data to the computer's permanent storage (disk drive or equivalent) ordinarily meets this requirement. In fact, even if a computer is fatally damaged, if the disk drives survive they can be moved to another computer with similar hardware and all committed transactions will remain intact.

While forcing data to the disk platters periodically might seem like a simple operation, it is not. Because disk drives are dramatically slower than main memory and CPUs, several layers of caching exist between the computer's main memory and the disk platters. First, there is the operating system's buffer cache, which caches frequently requested disk blocks and combines disk writes. Fortunately, all operating systems give applications a way to force writes from the buffer cache to disk, and PostgreSQL uses those features. (See the wal_sync_method parameter to adjust how this is done.)

Next, there might be a cache in the disk drive controller; this is particularly common on RAID controller cards. Some of these caches are write-through , meaning writes are sent to the drive as soon as they arrive. Others are write-back , meaning data is sent to the drive at some later time. Such caches can be a reliability hazard because the memory in the disk controller cache is volatile, and will lose its contents in a power failure. Better controller cards have battery-backup units (BBU s), meaning the card has a battery that maintains power to the cache in case of system power loss. After power is restored the data will be written to the disk drives.

And finally, most disk drives have caches. Some are write-through while some are write-back, and the same concerns about data loss exist for write-back drive caches as for disk controller caches. Consumer-grade IDE and SATA drives are particularly likely to have write-back caches that will not survive a power failure. Many solid-state drives (SSD) also have volatile write-back caches.

These caches can typically be disabled; however, the method for doing this varies by operating system and drive type:

• On Linux , IDE and SATA drives can be queried using hdparm -I ; write caching is enabled if there is a * next to Write cache . hdparm -W 0 can be used to turn off write caching. SCSI drives can be queried using sdparm . Use sdparm --get=WCE to check whether the write cache is enabled and sdparm --clear=WCE to disable it.

• On FreeBSD , IDE drives can be queried using camcontrol identify and write caching turned off using hw.ata.wc=0 in /boot/loader.conf ; SCSI drives can be queried using camcontrol identify , and the write cache both queried and changed using sdparm when available.

• On Solaris , the disk write cache is controlled by format -e . (The Solaris ZFS file system is safe with disk write-cache enabled because it issues its own disk cache flush commands.)

• On Windows , if wal_sync_method is open_datasync (the default), write caching can be disabled by unchecking My Computer\Open\ disk drive \Properties\Hardware\Properties\Policies\Enable write caching on the disk . Alternatively, set wal_sync_method to fdatasync (NTFS only), fsync or fsync_writethrough , which prevent write caching.

• On macOS , write caching can be prevented by setting wal_sync_method to fsync_writethrough .

Recent SATA drives (those following ATAPI-6 or later) offer a drive cache flush command (FLUSH CACHE EXT), while SCSI drives have long supported a similar command SYNCHRONIZE CACHE . These commands are not directly accessible to PostgreSQL , but some file systems (e.g., ZFS , ext4) can use them to flush data to the platters on write-back-enabled drives. Unfortunately, such file systems behave suboptimally when combined with battery-backup unit (BBU) disk controllers. In such setups, the synchronize command forces all data from the controller cache to the disks, eliminating much of the benefit of the BBU. You can run the pg_test_fsync program to see if you are affected. If you are affected, the performance benefits of the BBU can be regained by turning off write barriers in the file system or reconfiguring the disk controller, if that is an option. If write barriers are turned off, make sure the battery remains functional; a faulty battery can potentially lead to data loss. Hopefully file system and disk controller designers will eventually address this suboptimal behavior.

When the operating system sends a write request to the storage hardware, there is little it can do to make sure the data has arrived at a truly non-volatile storage area. Rather, it is the administrator's responsibility to make certain that all storage components ensure integrity for both data and file-system metadata. Avoid disk controllers that have non-battery-backed write caches. At the drive level, disable write-back caching if the drive cannot guarantee the data will be written before shutdown. If you use SSDs, be aware that many of these do not honor cache flush commands by default. You can test for reliable I/O subsystem behavior using diskchecker.pl .

Another risk of data loss is posed by the disk platter write operations themselves. Disk platters are divided into sectors, commonly 512 bytes each. Every physical read or write operation processes a whole sector. When a write request arrives at the drive, it might be for some multiple of 512 bytes (PostgreSQL typically writes 8192 bytes, or 16 sectors, at a time), and the process of writing could fail due to power loss at any time, meaning some of the 512-byte sectors were written while others were not. To guard against such failures, PostgreSQL periodically writes full page images to permanent WAL storage before modifying the actual page on disk. By doing this, during crash recovery PostgreSQL can restore partially-written pages from WAL. If you have file-system software that prevents partial page writes (e.g., ZFS), you can turn off this page imaging by turning off the full_page_writes parameter. Battery-Backed Unit (BBU) disk controllers do not prevent partial page writes unless they guarantee that data is written to the BBU as full (8kB) pages.

PostgreSQL also protects against some kinds of data corruption on storage devices that may occur because of hardware errors or media failure over time, such as reading/writing garbage data.

• Each individual record in a WAL file is protected by a CRC-32C (32-bit) check that allows us to tell if record contents are correct. The CRC value is set when we write each WAL record and checked during crash recovery, archive recovery and replication.

• Data pages are not currently checksummed by default, though full page images recorded in WAL records will be protected; see initdb for details about enabling data checksums.

• Internal data structures such as pg_xact , pg_subtrans , pg_multixact , pg_serial , pg_notify , pg_stat , pg_snapshots are not directly checksummed, nor are pages protected by full page writes. However, where such data structures are persistent, WAL records are written that allow recent changes to be accurately rebuilt at crash recovery and those WAL records are protected as discussed above.

• Individual state files in pg_twophase are protected by CRC-32C.

• Temporary data files used in larger SQL queries for sorts, materializations and intermediate results are not currently checksummed, nor will WAL records be written for changes to those files.

PostgreSQL does not protect against correctable memory errors and it is assumed you will operate using RAM that uses industry standard Error Correcting Codes (ECC) or better protection.

30.3. Write-Ahead Logging (WAL)

Write-Ahead Logging (WAL) is a standard method for ensuring data integrity. A detailed description can be found in most (if not all) books about transaction processing. Briefly, WAL 's central concept is that changes to data files (where tables and indexes reside) must be written only after those changes have been logged, that is, after WAL records describing the changes have been flushed to permanent storage. If we follow this procedure, we do not need to flush data pages to disk on every transaction commit, because we know that in the event of a crash we will be able to recover the database using the log: any changes that have not been applied to the data pages can be redone from the WAL records. (This is roll-forward recovery, also known as REDO.)

Tip

Because WAL restores database file contents after a crash, journaled file systems are not necessary for reliable storage of the data files or WAL files. In fact, journaling overhead can reduce performance, especially if journaling causes file system data to be flushed to disk. Fortunately, data flushing during journaling can often be disabled with a file system mount option, e.g., data=writeback on a Linux ext3 file system. Journaled file systems do improve boot speed after a crash.

Using WAL results in a significantly reduced number of disk writes, because only the WAL file needs to be flushed to disk to guarantee that a transaction is committed, rather than every data file changed by the transaction. The WAL file is written sequentially, and so the cost of syncing the WAL is much less than the cost of flushing the data pages. This is especially true for servers handling many small transactions touching different parts of the data store. Furthermore, when the server is processing many small concurrent transactions, one fsync of the WAL file may suffice to commit many transactions.

WAL also makes it possible to support on-line backup and point-in-time recovery, as described in Section 26.3 . By archiving the WAL data we can support reverting to any time instant covered by the available WAL data: we simply install a prior physical backup of the database, and replay the WAL just as far as the desired time. What's more, the physical backup doesn't have to be an instantaneous snapshot of the database state — if it is made over some period of time, then replaying the WAL for that period will fix any internal inconsistencies.

30.4. Asynchronous Commit

Asynchronous commit is an option that allows transactions to complete more quickly, at the cost that the most recent transactions may be lost if the database should crash. In many applications this is an acceptable trade-off.

As described in the previous section, transaction commit is normally synchronous : the server waits for the transaction's WAL records to be flushed to permanent storage before returning a success indication to the client. The client is therefore guaranteed that a transaction reported to be committed will be preserved, even in the event of a server crash immediately after. However, for short transactions this delay is a major component of the total transaction time. Selecting asynchronous commit mode means that the server returns success as soon as the transaction is logically completed, before the WAL records it generated have actually made their way to disk. This can provide a significant boost in throughput for small transactions.

Asynchronous commit introduces the risk of data loss. There is a short time window between the report of transaction completion to the client and the time that the transaction is truly committed (that is, it is guaranteed not to be lost if the server crashes). Thus asynchronous commit should not be used if the client will take external actions relying on the assumption that the transaction will be remembered. As an example, a bank would certainly not use asynchronous commit for a transaction recording an ATM's dispensing of cash. But in many scenarios, such as event logging, there is no need for a strong guarantee of this kind.

The risk that is taken by using asynchronous commit is of data loss, not data corruption. If the database should crash, it will recover by replaying WAL up to the last record that was flushed. The database will therefore be restored to a self-consistent state, but any transactions that were not yet flushed to disk will not be reflected in that state. The net effect is therefore loss of the last few transactions. Because the transactions are replayed in commit order, no inconsistency can be introduced — for example, if transaction B made changes relying on the effects of a previous transaction A, it is not possible for A's effects to be lost while B's effects are preserved.

The user can select the commit mode of each transaction, so that it is possible to have both synchronous and asynchronous commit transactions running concurrently. This allows flexible trade-offs between performance and certainty of transaction durability. The commit mode is controlled by the user-settable parameter synchronous_commit , which can be changed in any of the ways that a configuration parameter can be set. The mode used for any one transaction depends on the value of synchronous_commit when transaction commit begins.

Certain utility commands, for instance DROP TABLE , are forced to commit synchronously regardless of the setting of synchronous_commit . This is to ensure consistency between the server's file system and the logical state of the database. The commands supporting two-phase commit, such as PREPARE TRANSACTION , are also always synchronous.

If the database crashes during the risk window between an asynchronous commit and the writing of the transaction's WAL records, then changes made during that transaction will be lost. The duration of the risk window is limited because a background process (the “ WAL writer ”) flushes unwritten WAL records to disk every wal_writer_delay milliseconds. The actual maximum duration of the risk window is three times wal_writer_delay because the WAL writer is designed to favor writing whole pages at a time during busy periods.

Caution

An immediate-mode shutdown is equivalent to a server crash, and will therefore cause loss of any unflushed asynchronous commits.

Asynchronous commit provides behavior different from setting fsync = off. fsync is a server-wide setting that will alter the behavior of all transactions. It disables all logic within PostgreSQL that attempts to synchronize writes to different portions of the database, and therefore a system crash (that is, a hardware or operating system crash, not a failure of PostgreSQL itself) could result in arbitrarily bad corruption of the database state. In many scenarios, asynchronous commit provides most of the performance improvement that could be obtained by turning off fsync , but without the risk of data corruption.

commit_delay also sounds very similar to asynchronous commit, but it is actually a synchronous commit method (in fact, commit_delay is ignored during an asynchronous commit). commit_delay causes a delay just before a transaction flushes WAL to disk, in the hope that a single flush executed by one such transaction can also serve other transactions committing at about the same time. The setting can be thought of as a way of increasing the time window in which transactions can join a group

about to participate in a single flush, to amortize the cost of the flush among multiple transactions.

30.5. WAL Configuration

There are several WAL -related configuration parameters that affect database performance. This section explains their use. Consult Chapter 20 for general information about setting server configuration parameters.

Checkpoints are points in the sequence of transactions at which it is guaranteed that the heap and index data files have been updated with all information written before that checkpoint. At checkpoint time, all dirty data pages are flushed to disk and a special checkpoint record is written to the WAL file. (The change records were previously flushed to the WAL files.) In the event of a crash, the crash recovery procedure looks at the latest checkpoint record to determine the point in the WAL (known as the redo record) from which it should start the REDO operation. Any changes made to data files before that point are guaranteed to be already on disk. Hence, after a checkpoint, WAL segments preceding the one containing the redo record are no longer needed and can be recycled or removed. (When WAL archiving is being done, the WAL segments must be archived before being recycled or removed.)

The checkpoint requirement of flushing all dirty data pages to disk can cause a significant I/O load. For this reason, checkpoint activity is throttled so that I/O begins at checkpoint start and completes before the next checkpoint is due to start; this minimizes performance degradation during checkpoints.

The server's checkpointer process automatically performs a checkpoint every so often. A checkpoint is begun every checkpoint_timeout seconds, or if max_wal_size is about to be exceeded, whichever comes first. The default settings are 5 minutes and 1 GB, respectively. If no WAL has been written since the previous checkpoint, new checkpoints will be skipped even if checkpoint_timeout has passed. (If WAL archiving is being used and you want to put a lower limit on how often files are archived in order to bound potential data loss, you should adjust the archive_timeout parameter rather than the checkpoint parameters.) It is also possible to force a checkpoint by using the SQL command CHECKPOINT .

Reducing checkpoint_timeout and/or max_wal_size causes checkpoints to occur more often. This allows faster after-crash recovery, since less work will need to be redone. However, one must balance this against the increased cost of flushing dirty data pages more often. If full_page_writes is set (as is the default), there is another factor to consider. To ensure data page consistency, the first modification of a data page after each checkpoint results in logging the entire page content. In that case, a smaller checkpoint interval increases the volume of output to the WAL, partially negating the goal of using a smaller interval, and in any case causing more disk I/O.

Checkpoints are fairly expensive, first because they require writing out all currently dirty buffers, and second because they result in extra subsequent WAL traffic as discussed above. It is therefore wise to set the checkpointing parameters high enough so that checkpoints don't happen too often. As a simple sanity check on your checkpointing parameters, you can set the checkpoint_warning parameter. If checkpoints happen closer together than checkpoint_warning seconds, a message will be output to the server log recommending increasing max_wal_size . Occasional appearance of such a message is not cause for alarm, but if it appears often then the checkpoint control parameters should be increased. Bulk operations such as large COPY transfers might cause a number of such warnings to appear if you have not set max_wal_size high enough.

To avoid flooding the I/O system with a burst of page writes, writing dirty buffers during a checkpoint is spread over a period of time. That period is controlled by checkpoint_completion_target , which is given as a fraction of the checkpoint interval (configured by using checkpoint_timeout). The I/O rate is

adjusted so that the checkpoint finishes when the given fraction of checkpoint_timeout seconds have elapsed, or before max_wal_size is exceeded, whichever is sooner. With the default value of 0.9, PostgreSQL can be expected to complete each checkpoint a bit before the next scheduled checkpoint (at around 90% of the last checkpoint's duration). This spreads out the I/O as much as possible so that the checkpoint I/O load is consistent throughout the checkpoint interval. The disadvantage of this is that prolonging checkpoints affects recovery time, because more WAL segments will need to be kept around for possible use in recovery. A user concerned about the amount of time required to recover might wish to reduce checkpoint_timeout so that checkpoints occur more frequently but still spread the I/O across the checkpoint interval. Alternatively, checkpoint_completion_target could be reduced, but this would result in times of more intense I/O (during the checkpoint) and times of less I/O (after the checkpoint completed but before the next scheduled checkpoint) and therefore is not recommended. Although checkpoint_completion_target could be set as high as 1.0, it is typically recommended to set it to no higher than 0.9 (the default) since checkpoints include some other activities besides writing dirty buffers. A setting of 1.0 is quite likely to result in checkpoints not being completed on time, which would result in performance loss due to unexpected variation in the number of WAL segments needed.

On Linux and POSIX platforms checkpoint_flush_after allows to force the OS that pages written by the checkpoint should be flushed to disk after a configurable number of bytes. Otherwise, these pages may be kept in the OS's page cache, inducing a stall when fsync is issued at the end of a checkpoint. This setting will often help to reduce transaction latency, but it also can have an adverse effect on performance; particularly for workloads that are bigger than shared_buffers , but smaller than the OS's page cache.

The number of WAL segment files in pg_wal directory depends on min_wal_size , max_wal_size and the amount of WAL generated in previous checkpoint cycles. When old WAL segment files are no longer needed, they are removed or recycled (that is, renamed to become future segments in the numbered sequence). If, due to a short-term peak of WAL output rate, max_wal_size is exceeded, the unneeded segment files will be removed until the system gets back under this limit. Below that limit, the system recycles enough WAL files to cover the estimated need until the next checkpoint, and removes the rest. The estimate is based on a moving average of the number of WAL files used in previous checkpoint cycles. The moving average is increased immediately if the actual usage exceeds the estimate, so it accommodates peak usage rather than average usage to some extent. min_wal_size puts a minimum on the amount of WAL files recycled for future usage; that much WAL is always recycled for future use, even if the system is idle and the WAL usage estimate suggests that little WAL is needed.

Independently of max_wal_size , the most recent wal_keep_size megabytes of WAL files plus one additional WAL file are kept at all times. Also, if WAL archiving is used, old segments cannot be removed or recycled until they are archived. If WAL archiving cannot keep up with the pace that WAL is generated, or if archive_command or archive_library fails repeatedly, old WAL files will accumulate in pg_wal until the situation is resolved. A slow or failed standby server that uses a replication slot will have the same effect (see Section 27.2.6).

In archive recovery or standby mode, the server periodically performs restartpoints , which are similar to checkpoints in normal operation: the server forces all its state to disk, updates the pg_control file to indicate that the already-processed WAL data need not be scanned again, and then recycles any old WAL segment files in the pg_wal directory. Restartpoints can't be performed more frequently than checkpoints on the primary because restartpoints can only be performed at checkpoint records. A restartpoint is triggered when a checkpoint record is reached if at least checkpoint_timeout seconds have passed since the last restartpoint, or if WAL size is about to exceed max_wal_size . However, because of limitations on when a restartpoint can be performed, max_wal_size is often exceeded during recovery, by up to one checkpoint cycle's worth of WAL. (max_wal_size is never a hard limit anyway, so you should always leave plenty of headroom to avoid running out of disk space.)

There are two commonly used internal WAL functions: XLogInsertRecord and XLogFlush . XLogInsertRecord is used to place a new record into the WAL buffers in shared memory. If there is no space for the new record, XLogInsertRecord will have to write (move to kernel cache) a few filled WAL buffers. This is undesirable because XLogInsertRecord is used on every database low level modification (for example, row insertion) at a time when an exclusive lock is held on affected data pages, so the operation needs to be as fast as possible. What is worse, writing WAL buffers might also force the creation of a new WAL segment, which takes even more time. Normally, WAL buffers should be written and flushed by an XLogFlush request, which is made, for the most part, at transaction commit time to ensure that transaction records are flushed to permanent storage. On systems with high WAL output, XLogFlush requests might not occur often enough to prevent XLogInsertRecord from having to do writes. On such systems one should increase the number of WAL buffers by modifying the wal_buffers parameter. When full_page_writes is set and the system is very busy, setting wal_buffers higher will help smooth response times during the period immediately following each checkpoint.

The commit_delay parameter defines for how many microseconds a group commit leader process will sleep after acquiring a lock within XLogFlush , while group commit followers queue up behind the leader. This delay allows other server processes to add their commit records to the WAL buffers so that all of them will be flushed by the leader's eventual sync operation. No sleep will occur if fsync is not enabled, or if fewer than commit_siblings other sessions are currently in active transactions; this avoids sleeping when it's unlikely that any other session will commit soon. Note that on some platforms, the resolution of a sleep request is ten milliseconds, so that any nonzero commit_delay setting between 1 and 10000 microseconds would have the same effect. Note also that on some platforms, sleep operations may take slightly longer than requested by the parameter.

Since the purpose of commit_delay is to allow the cost of each flush operation to be amortized across concurrently committing transactions (potentially at the expense of transaction latency), it is necessary to quantify that cost before the setting can be chosen intelligently. The higher that cost is, the more effective commit_delay is expected to be in increasing transaction throughput, up to a point. The pg_test_fsync program can be used to measure the average time in microseconds that a single WAL flush operation takes. A value of half of the average time the program reports it takes to flush after a single 8kB write operation is often the most effective setting for commit_delay , so this value is recommended as the starting point to use when optimizing for a particular workload. While tuning commit_delay is particularly useful when the WAL is stored on high-latency rotating disks, benefits can be significant even on storage media with very fast sync times, such as solid-state drives or RAID arrays with a battery-backed write cache; but this should definitely be tested against a representative workload. Higher values of commit_siblings should be used in such cases, whereas smaller commit_siblings values are often helpful on higher latency media. Note that it is quite possible that a setting of commit_delay that is too high can increase transaction latency by so much that total transaction throughput suffers.

When commit_delay is set to zero (the default), it is still possible for a form of group commit to occur, but each group will consist only of sessions that reach the point where they need to flush their commit records during the window in which the previous flush operation (if any) is occurring. At higher client counts a “ gangway effect ” tends to occur, so that the effects of group commit become significant even when commit_delay is zero, and thus explicitly setting commit_delay tends to help less. Setting commit_delay can only help when (1) there are some concurrently committing transactions, and (2) throughput is limited to some degree by commit rate; but with high rotational latency this setting can be effective in increasing transaction throughput with as few as two clients (that is, a single committing client with one sibling transaction).

The wal_sync_method parameter determines how PostgreSQL will ask the kernel to force WAL updates out to disk. All the options should be the same in terms of reliability, with the exception of fsync_writethrough , which can sometimes force a flush of the disk cache even when other options do

not do so. However, it's quite platform-specific which one will be the fastest. You can test the speeds of different options using the pg_test_fsync program. Note that this parameter is irrelevant if fsync has been turned off.

Enabling the wal_debug configuration parameter (provided that PostgreSQL has been compiled with support for it) will result in each XLogInsertRecord and XLogFlush WAL call being logged to the server log. This option might be replaced by a more general mechanism in the future.

There are two internal functions to write WAL data to disk: XLogWrite and issue_xlog_fsync . When track_wal_io_timing is enabled, the total amounts of time XLogWrite writes and issue_xlog_fsync syncs WAL data to disk are counted as wal_write_time and wal_sync_time in pg_stat_wal , respectively. XLogWrite is normally called by XLogInsertRecord (when there is no space for the new record in WAL buffers), XLogFlush and the WAL writer, to write WAL buffers to disk and call issue_xlog_fsync . issue_xlog_fsync is normally called by XLogWrite to sync WAL files to disk. If wal_sync_method is either open_datasync or open_sync , a write operation in XLogWrite guarantees to sync written WAL data to disk and issue_xlog_fsync does nothing. If wal_sync_method is either fdatasync , fsync , or fsync_writethrough , the write operation moves WAL buffers to kernel cache and issue_xlog_fsync syncs them to disk. Regardless of the setting of track_wal_io_timing , the number of times XLogWrite writes and issue_xlog_fsync syncs WAL data to disk are also counted as wal_write and wal_sync in pg_stat_wal , respectively.

The recovery_prefetch parameter can be used to reduce I/O wait times during recovery by instructing the kernel to initiate reads of disk blocks that will soon be needed but are not currently in PostgreSQL 's buffer pool. The maintenance_io_concurrency and wal_decode_buffer_size settings limit prefetching concurrency and distance, respectively. By default, it is set to try , which enables the feature on systems where posix_fadvise is available.